LOW-LEVEL DESIGN (LLD)

RAG-BASED CUSTOMER SUPPORT ASSISTANT

(Retrieval-Augmented Generation with Workflow Routing & Human-in-the-Loop)

Submitted by:
Sai Kadiravan S Intern ID : IN126008102

Institution:
Innomatics Research Labs

Program:
Computer Science Engineering

Project Type:
AI / Machine Learning Internship Project

Knowledge Base Used:
Customer Service Handbook (PDF containing customer interaction guidelines, complaint handling procedures, and service policies)

Technologies Used:
Python, LangChain, ChromaDB, Groq LLM, HuggingFace Embeddings

Submission Date:
23/04/2026

.

TABLE OF CONTENTS

1. Module-Level Design
    - 1.1 Document Processing Module
    - 1.2 Chunking Module
    - 1.3 Embedding Module
    - 1.4 Vector Storage Module
    - 1.5 Retrieval Module
    - 1.6 Query Processing Module
    - 1.7 Graph Execution Module
    - 1.8 HITL Module
2. Data Structures
    - 2.1 Document Representation
    - 2.2 Chunk Format
    - 2.3 Embedding Structure
    - 2.4 Query-Response Schema
    - 2.5 State Object for Graph
3. Workflow Design (LangGraph)
    - 3.1 Nodes
    - 3.2 Edges
    - 3.3 State Flow Between Nodes
4. Conditional Routing Logic
    - 4.1 Answer Generation Criteria
    - 4.2 Escalation Criteria
5. HITL Design
    - 5.1 When Escalation is Triggered
    - 5.2 What Happens After Escalation
    - 5.3 Human Response Integration
6. API / Interface Design
    - 6.1 Input Format
    - 6.2 Output Format
    - 6.3 Interaction Flow
7. Error Handling
    - 7.1 Missing Data / PDF Not Found
    - 7.2 No Relevant Chunks Found
    - 7.3 LLM Failure

1. Module-Level Design

1.1 Document Processing Module

File: ingestion.py

Responsible for loading the raw PDF from disk. Uses PyPDFLoader which reads each page as a separate LangChain Document object, attaching metadata (source path, page number). Output is a list of Document objects passed directly to the Chunking Module.

Key Specifications
• Input: PDF file path (str)
• Output: List[Document] — one per page
• Key dependency: langchain_community.document_loaders.PyPDFLoader
• Error handling: FileNotFoundError if PDF path is invalid

1.2 Chunking Module

File: ingestion.py

Splits Document objects into smaller, semantically coherent chunks using RecursiveCharacterTextSplitter. The recursive strategy tries to split on paragraph breaks, then sentences, then words — preserving natural language boundaries.

Key Specifications
• chunk_size = 800 characters (balances context richness vs retrieval noise)
• chunk_overlap = 100 characters (prevents boundary information loss)
• Input: List[Document] Output: List[Document] (chunks)
• Each chunk inherits source metadata from its parent page

1.3 Embedding Module

File: ingestion.py + retriever.py

Converts text chunks into dense numerical vectors using HuggingFace's all-MiniLM-L6-v2 sentence transformer. The same model instance is used for both ingestion (chunk embedding) and retrieval (query embedding) to ensure vector space consistency.

Key Specifications
• Model: all-MiniLM-L6-v2 | Output dimensions: 384
• No API key required — model weights downloaded once and cached locally
• Similarity metric: cosine similarity (ChromaDB default)
• Input: str or List[str] Output: List[float] (384-dim vector)

1.4 Vector Storage Module

File: ingestion.py + retriever.py

Manages persistence of chunk embeddings in ChromaDB. On first run, creates the collection at ./chroma_db. On subsequent runs, loads the existing collection. Supports add, query, and delete operations.

Key Specifications
• Persist directory: ./chroma_db
• Collection auto-created on first Chroma.from_documents() call
• Existence check: os.path.exists(CHROMA_DIR) and os.listdir(CHROMA_DIR)
• Storage format: SQLite (metadata) + HNSW index (vectors)

1.5 Retrieval Module

File: retriever.py

Wraps ChromaDB as a LangChain BaseRetriever. Accepts a query string, embeds it using the same embedding model, and performs approximate nearest-neighbour search returning the top-k most similar chunks.

Key Specifications
• k = 3 chunks returned per query (configurable)
• Method: vectorstore.as_retriever(search_kwargs={'k': 3})
• Input: query (str) Output: List[Document]
• Retrieval type: similarity search (cosine distance)

1.6 Query Processing Module

File: graph.py — generate_node()

Formats the retrieved chunks into a structured prompt and invokes the Groq LLM. The prompt is deliberately constrained: the model is instructed to answer only from the provided context, minimising hallucination.

Key Specifications
• Prompt template: System role + Context injection + User question
• LLM: ChatGroq with model llama-3.3-70b-versatile
• Input: query (str) + context (List[str]) Output: response (str)
• Fallback instruction: 'Say I don't know if answer not in context'

1.7 Graph Execution Module

File: graph.py — build_graph()

Constructs and compiles the LangGraph StateGraph. Registers nodes, defines directed edges, sets the entry point, and returns a compiled app object. The compiled app is stateless between invocations — state is passed per call.

Key Specifications
• Entry point: retrieve node (set via set_entry_point)
• Edges: retrieve→generate→route→END (all deterministic)
• State type: SupportState (TypedDict)
• Compilation: workflow.compile() returns a Pregel-based runnable

1.8 HITL Module

File: hitl.py

Implements the Human-in-the-Loop escalation logic. Contains two functions: should_escalate() which evaluates escalation criteria and returns a boolean, and human_handoff() which simulates a live agent response via CLI stdin.

Key Specifications
• should_escalate(): keyword match + uncertainty detection + length check
• human_handoff(): prints escalation notice, accepts agent input via input()
• Input to should_escalate: query (str), response (str), context (list)
• Output of human_handoff: agent_response (str)

2. Data Structures

2.1 Document Representation

Each page loaded from the PDF is represented as a LangChain Document object:

```
class Document:
    page_content: str        # raw text of the page
    metadata: dict           # e.g. {'source': 'data/handbook.pdf', 'page': 3}
```

2.2 Chunk Format

After splitting, each chunk is also a Document. The metadata is inherited from the parent page:

```
chunk = Document(
    page_content = 'Customer complaints should be acknowledged within...'
    metadata     = {'source': 'data/handbook.pdf', 'page': 7}
)
# chunk_size=800, chunk_overlap=100 — approx 130-160 words per chunk
```

2.3 Embedding Structure

Each chunk is converted to a 384-dimensional float vector before storage in ChromaDB:

```
embedding_vector: List[float]    # length = 384
# Stored in ChromaDB alongside:
#   document   : str    (the chunk text)
#   metadata   : dict   (source, page)
#   id         : str    (auto-generated UUID)
```

2.4 Query-Response Schema

The prompt sent to the LLM follows this structure, and the response is plain text:

```
# Prompt (constructed in generate_node)
prompt = f'''
You are a helpful customer support assistant.
Use ONLY the context below to answer the question.
If the answer is not available, say 'I don't know.'
Context: {context_text}
Question: {state['query']}
Answer:'''
# Response
response: str   # plain text answer from LLaMA 3.3 70B
```

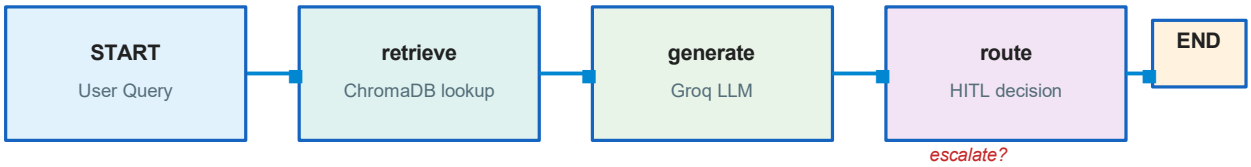
2.5 State Object for Graph

SupportState is a TypedDict that carries all data between LangGraph nodes:

```
class SupportState(TypedDict):
    query         : str    # original user input
    context       : list   # List[str] — top-k retrieved chunk texts
    response      : str    # LLM-generated answer (before routing)
```

```
    escalated    : bool   # True if HITL was triggered
    final_answer : str    # what is shown to the user
# Initial invocation always passes all keys:
app.invoke({'query': q, 'context': [], 'response': '',
            'escalated': False, 'final_answer': ''})
```

3. Workflow Design (LangGraph)



SupportState flows through every node → query | context | response | escalated | final_answer

Figure 2 — LangGraph StateGraph: deterministic edges connect retrieve → generate → route → END. SupportState is passed through every node.

3.1 Nodes

Node	Function	Input Keys Read	Output Keys Written
retrieve	`retrieve_node()`	query	context
generate	`generate_node()`	query, context	response
route	`route_node()`	query, response, context	escalated, final_answer

3.2 Edges

- START → retrieve (set via workflow.set_entry_point('retrieve'))
- retrieve → generate (workflow.add_edge('retrieve', 'generate'))
- generate → route (workflow.add_edge('generate', 'route'))
- route → END (workflow.add_edge('route', END))

All edges are deterministic — there are no conditional edges in this graph. Branching logic is handled inside the route node itself, keeping the graph structure simple and the decision logic encapsulated in the HITL module.

3.3 State Flow Between Nodes

```
# State after retrieve_node:
{'query': 'How to handle returns?', 'context': ['chunk1...', 'chunk2...', 'chunk3...'],
 'response': '', 'escalated': False, 'final_answer': ''}
# State after generate_node:
{'query': 'How to handle returns?', 'context': [...],
 'response': 'Returns must be initiated within 30 days...', 'escalated': False, ...}
# State after route_node (no escalation):
{'query': '...', 'context': [...], 'response': '...',
 'escalated': False, 'final_answer': 'Returns must be initiated within 30 days...'}
```

4. Conditional Routing Logic

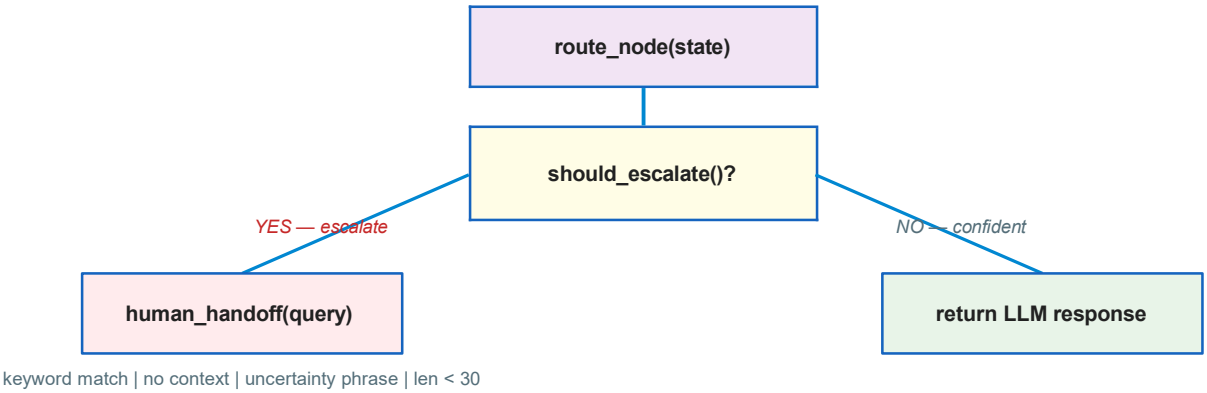


Figure 3 — should_escalate() decision tree: four independent criteria, any one sufficient to trigger HITL.

4.1 Answer Generation Criteria (NO escalation)

The LLM answer is returned directly to the user when ALL of the following hold:

- ✓ No escalation keywords found in the user query.
- ✓ At least one relevant chunk was retrieved from ChromaDB.
- ✓ The LLM response does not contain uncertainty phrases.
- ✓ The LLM response is longer than 30 characters.

4.2 Escalation Criteria (triggers HITL)

Keyword Match

Trigger	Reason
complaint, issue, problem, broken, refund, damaged, not working, angry	The query directly signals customer dissatisfaction. Human empathy and authority are needed.

Missing Context

Trigger	Reason
context_docs list is empty after retrieval	ChromaDB returned no relevant chunks. The PDF knowledge base does not contain an answer — a human must respond.

Low Confidence / Uncertainty

Trigger	Reason
Response contains: 'i don't know', 'i'm not sure', 'cannot find', 'not mentioned', 'unclear'	The LLM signals it could not find the answer in context. Rather than hallucinate, it escalates.

Trivial / Failed Response

Trigger	Reason
len(response.strip()) < 30 characters	A very short response typically indicates the LLM produced a non-answer. Treated as low confidence.

```
# should_escalate() implementation logic:
def should_escalate(query, response, context_docs) -> bool:
    if any(word in query.lower() for word in ESCALATION_KEYWORDS): return True
    if not context_docs:                                            return True
    if any(p in response.lower() for p in UNCERTAINTY_PHRASES):    return True
    if len(response.strip()) < 30:                                  return True
    return False
```

5. HITL Design

5.1 When Escalation is Triggered

Escalation is evaluated inside route_node() after generate_node() completes. The route node calls should_escalate() with three arguments: the original query, the LLM-generated response, and the list of retrieved context documents. If any criterion returns True, the escalation path is taken immediately — the LLM response is discarded and human_handoff() is called.

5.2 What Happens After Escalation

A visual separator and escalation notice are printed to the terminal.

The original customer query is displayed for the human agent to review.

The agent types their response at the 'Human Agent Response:' prompt.

The agent's input is captured as a string via Python's built-in input().

The string is stored in state['final_answer'] and state['escalated'] = True.

The graph proceeds to END — the agent response is returned to the CLI loop.

The CLI displays the response prefixed with [Human Agent] instead of [Bot].

5.3 How Human Response is Integrated

```
# Inside route_node() — escalation branch:
if escalate:
    human_reply = human_handoff(state['query'])    # blocks for agent input
    return {
        **state,
        'escalated'     : True,
        'final_answer' : human_reply               # replaces LLM response
    }
# In main.py — display logic:
if result['escalated']:
    print(f'[Human Agent]: {result["final_answer"]}')
else:
    print(f'[Bot]: {result["final_answer"]}')
```

Note: In a production system, human_handoff() would be replaced by a webhook call to a ticketing platform (Zendesk, Freshdesk), publishing the escalated query as a new ticket and asynchronously waiting for the agent response. The CLI implementation serves as a functional simulation of this pattern for demonstration purposes.

6. API / Interface Design

6.1 Input Format

The system accepts input via the CLI REPL in main.py. Internally, the LangGraph app.invoke() call expects a fully initialised SupportState dictionary:

```
# CLI input
query: str = input('You: ').strip()
# LangGraph invocation input
app.invoke({
    'query'        : query,    # str — user's natural language question
    'context'      : [],       # list — initialised empty, filled by retrieve_node
    'response'     : '',       # str  — initialised empty, filled by generate_node
    'escalated'    : False,    # bool — initialised False, set by route_node
    'final_answer' : ''        # str  — initialised empty, set by route_node
})
```

6.2 Output Format

app.invoke() returns the final SupportState. The CLI reads two keys:

```
result: SupportState = app.invoke({...})
result['escalated']     : bool   # True → human answered, False → bot answered
result['final_answer'] : str    # the answer to display to the user
# Display
prefix = '[Human Agent]' if result['escalated'] else '[Bot]'
print(f'{prefix}: {result["final_answer"]}')
```

6.3 Interaction Flow

Step	Actor	Action	System Response
1	User	Types a question at CLI	Input captured as string
2	System	app.invoke() called with initial state	LangGraph begins execution
3	retrieve_node	Embeds query, queries ChromaDB	context list populated
4	generate_node	Builds prompt, calls Groq API	response string populated
5a	route_node	should_escalate() → False	final_answer = LLM response; printed to user
5b	route_node	should_escalate() → True	human_handoff() called; agent types response; printed to user

7. Error Handling

7.1 Missing Data / PDF Not Found

If the PDF file does not exist at the configured path, PyPDFLoader raises a FileNotFoundError. The ingestion module should be wrapped in a try/except block in production. In the current implementation, the os.path.exists(CHROMA_DIR) check in main.py prevents re-ingestion on subsequent runs — but the initial run will fail clearly if the PDF is absent.

Handling Details
• Detection: FileNotFoundError from PyPDFLoader
• Behaviour: Stack trace printed; application exits
• Recommended fix: Wrap ingest_pdf() in try/except; print actionable error message
• Prevention: Validate PDF path at startup before ingestion attempt

7.2 No Relevant Chunks Found

If ChromaDB returns an empty list for a query (no chunks meet the similarity threshold), the context list in SupportState will be empty. This is explicitly handled by the should_escalate() function which returns True when context_docs is empty, triggering HITL rather than allowing the LLM to answer without grounding.

Handling Details
• Detection: len(context_docs) == 0 inside should_escalate()
• Behaviour: HITL escalation triggered automatically
• User experience: Human agent responds instead of an empty or hallucinated answer
• No exception raised — handled gracefully through routing logic

7.3 LLM Failure (Groq API Error)

Network errors, API rate limits, or invalid API keys will cause the ChatGroq.invoke() call in generate_node() to raise an exception (typically a requests.exceptions or langchain_core APIError). This will propagate up through LangGraph and crash the loop.

Handling Details
• Detection: Exception from llm.invoke() in generate_node()
• Behaviour (current): Unhandled — loop crashes with traceback
• Recommended fix: Wrap llm.invoke() in try/except; set response to empty string on failure
• Fallback: Empty response triggers should_escalate() length check → HITL escalation

Recommended production-grade generate_node() with error handling:

```
def generate_node(state: SupportState) -> SupportState:
    try:
```

```
        result   = llm.invoke([HumanMessage(content=prompt)])
        response = result.content
    except Exception as e:
        print(f'LLM error: {e}')
        response = ''   # triggers HITL via length check in should_escalate()
    return {**state, 'response': response}
```